Module 04 - Prompt engineering to Product Requirement Document (PRD) - step by step

Highly practical, real-world skill: **using AI to accelerate product management and software architecture**.

📄 Assignment 04: The AI Product Manager
Course: Natural Language Processing (AI 2026)
Module: 04 - Prompt Engineering & In-Context Learning
Estimated Time: 1.5 - 2 Hours

Product Manager Prompts: Sample Product Manager PromptsLinks to an external site.

🎯 Objective
In this assignment, you will simulate the workflow of a modern Product Manager (PM). **You have a "Rough Idea" for a new application/Project/Service**. Your goal is to use **Prompt Engineering** to turn that vague idea into a professional, developer-ready **Product Requirement Document (PRD)** that you can use with GenAI code generation tools to build from that PRD.

Product Manager Prompts

CRYPTOCURRENCY SYSTEM USING BODY ACTIVITY DATA

Human body activity associated with a task provided to a user may be used in a mining process of a cryptocurrency system. A server may provide a task to a device of a user which is communicatively coupled to the server. A sensor communicatively coupled to or comprised in the device of the user may sense body activity of the user. Body activity data may be generated based on the sensed body activity of the user. The cryptocurrency system communicatively coupled to the device of the user may verify if the body activity data satisfies one or more conditions set by the cryptocurrency system, and award cryptocurrency to the user whose body activity data is verified.

You will learn why "asking nicely" fails, and how **Context**, **Examples**, and **Reasoning** produce enterprise-grade specifications.

🛠️ Scenario

CRYPTOCURRENCY SYSTEM USING BODY ACTIVITY DATA

BITWEAR
A decentralized cryptocurrency system where mining rewards are earned through sensor-verified human physical activity instead of computational waste.

Concept
A user is given a task through a device connected to a server. While the user performs the task, sensors on the device measure the user's body movements and activity. This activity data is then sent to a cryptocurrency system, which checks whether the task was completed correctly based on specific rules. If the activity meets the required conditions, the system rewards the user with cryptocurrency.

You are a PM at a startup. You have an idea for an app called ***BITWEAR***
user is given a task through a device connected to a server. While the user performs the task, sensors on the device measure the user's body movements and activity. This activity data is then sent to a cryptocurrency system, which checks whether the task was completed correctly based on specific rules. If the activity meets the required conditions, the system rewards the user with cryptocurrency.

Your developers are waiting or you yourself want to vibe code it. If you send them vague requirements, they will build the wrong thing. You **must Engineer the AI to write the specs** for you.

📝 Instructions

Task 1: The Zero-Shot Baseline (The "Lazy PM") First, try the naive approach.

Prompt Ask the model simply: *"Write a PRD for an app called BITWEAR where user is given a task through a device connected to a server. While the user performs the task, sensors on the device measure the user's body movements and activity. This activity data is then sent to a cryptocurrency system, which checks whether the task was completed correctly based on specific rules. If the activity meets the required conditions, the system rewards the user with cryptocurrency."*

Analyze Look at the output. Is it vague? Does it say things like "The app should be fast" or "The app should have a good UI"? (These are useless to developers).

Task 2: Few-Shot Prompting (The "Company Standard")
Your company has a specific format for writing **User Stories**. You need the AI to mimic this strict format.

* **Strategy:** Provide the AI with 2 examples of "Gold Standard" user stories from a different project (e.g., a Ride-Sharing app).

* **Format:**
  * *Example Input:* "Feature: Booking a Ride."
  * *Example Output:* "As a **Rider**, I want to **input my destination**, so that **I can get a fare estimate**. [Acceptance Criteria: Map loads within 2s, Fare visible before confirmation]."

* **Task:** Feed these examples in the prompt and ask it to generate 3 User Stories for **FridgeChef** (specifically for the "Camera/Scanning" feature).

Task 3: Structural Reasoning (Simulation before Specification)
A PRD fails when it misses "Edge Cases" (e.g., What if the user has no internet? What if the fridge is empty?).

* **Strategy:** Use **Chain-of-Thought** prompting.
* **The Prompt:** Do *not* ask for the PRD yet. Instead, command the AI to:
  1. `<simulation>`: Simulate a user trying to use the app in a **difficult** scenario (poor lighting, bad internet, unidentified vegetables). Write out the narrative of their struggle step-by-step.

  2. `<requirements>`: Based *only* on that simulation, list the technical requirements needed to solve those specific struggles (e.g., "Offline Mode," "Low-Light Image Enhancement").

* **Goal:** Force the model to "live" the problem before it solves it.

Task 4: The "Cynical CTO" Persona (Refinement)
LLMs are often too agreeable. You need a critic.
* **Strategy:** Prompt Chaining.
* **Step A:** Take the requirements from Task 3.
* **Step B:** Feed them into a new prompt with the persona: *"You are a cynical, grumpy Chief Technology Officer. Tear these requirements apart. Find security risks, privacy issues with uploading photos, and hallucination risks."*
* **Step C:** Ask the model to rewrite the Final PRD incorporating the CTO's feedback.

📦 Submission
Submit a Jupyter Notebook (`.ipynb`) containing:
1. **The Prompt Chains:** The specific text you sent to the model for each task.
2. **The Evolution:** A side-by-side comparison of the Task 1 (Lazy) output vs. the Task 4 (Refined) output.
3. **Reflection:** A short paragraph on which prompting technique (Few-Shot, CoT, Persona) added the most value to the document.

🚀 Instructor's Solution Notebook

This solution uses the "FridgeChef" example, but students can use their own ideas.

```python
# ========================================
**# MODULE 04 ASSIGNMENT: THE AI PRODUCT MANAGER SAMPLE**
# ========================================

# !pip install openai

import os
from openai import OpenAI

# Setup Client
# Assuming running locally or with an API key
client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))
```

```
def get_completion(prompt, model="gpt-4o"):
  response = client.chat.completions.create(
    model=model,
    messages=[{"role": "user", "content": prompt}],
    temperature=0.7 # Slight creativity allowed for brainstorming
  )
  return response.choices[0].message.content

# THE PRODUCT IDEA
product_idea = "FridgeChef: An app where you snap a pic of your open fridge, and AI suggests recipes based on ingredients found."

# ========================================
# TASK 1: ZERO-SHOT BASELINE (FAILURE MODE)
# ========================================
print("--- TASK 1: The Lazy PRD ---")

prompt_1 = f"Write a Product Requirement Document (PRD) for this app: {product_idea}"

response_1 = get_completion(prompt_1)
print(response_1[:500] + "...\n[TRUNCATED]")

# CRITIQUE:
# Usually produces generic headers like "Introduction", "Features".
# Requirements are vague: "Must be user friendly." "Must recognize food."
# Developers cannot code from this.


# ========================================
# TASK 2: FEW-SHOT PROMPTING (STANDARDIZATION)
# ========================================
print("\n--- TASK 2: Few-Shot User Stories ---")

# We teach the model the specific "Company Format" via examples
prompt_2 = f"""
I need to write User Stories for {product_idea}.
Adopt the following strict format based on these examples:

Example 1 (Ride Share App):
Input: Driver matching
Output: "As a **Rider**, I want to **see the driver's location in real-time**, so that **I feel safe waiting outside**. [Criteria: Update frequency < 3s, Car model displayed]."
```

Example 2 (Email App):
Input: Search
Output: "As a **User**, I want to **filter emails by attachment**, so that **I can find documents quickly**. [Criteria: Filter applies instantly, supports PDF/DOCX]."

Task:
Write 3 User Stories for FridgeChef focused on the "Camera Scanning" feature.
"""

response_2 = get_completion(prompt_2)
print(response_2)

OBSERVATION:
The model now outputs bolded personas, clear "so that" benefits, and specific acceptance criteria.

# ==
# TASK 3: STRUCTURAL REASONING (SIMULATION)
# ==
print("\n--- TASK 3: The Edge Case Simulation (Chain of Thought) ---")

prompt_3 = f"""
We are building {product_idea}.
Do NOT write the PRD yet.

Step 1: <simulation>
Simulate a user named 'Tired Tony'. It is 8 PM, his kitchen lighting is dim,
he has spotty WiFi, and his fridge is messy with a half-eaten rotisserie chicken
and some wilted kale. Narrate his attempt to use the app and the frustrations he faces.
</simulation>

Step 2: <requirements>
Based EXCLUSIVELY on Tony's struggle, list 4 technical requirements to solve his pain points.
(e.g., if lighting is bad, what tech do we need? If wifi is bad, how do we handle image upload?)
</requirements>
"""

response_3 = get_completion(prompt_3)
print(response_3)

```
# OBSERVATION:
# The simulation forces the AI to "realize" that low-light enhancement and
# client-side compression (for bad wifi) are necessary features.
# A zero-shot prompt usually misses these edge cases.


# ========================================
# TASK 4: THE CYNICAL CTO (REFINEMENT)
# ========================================
print("\n--- TASK 4: The Final Polish ---")

# We pass the output of Task 3 into Task 4
previous_requirements = response_3

prompt_4 = f"""
You are a grumpy, paranoid CTO who cares deeply about Security, Privacy, and Hallucinations.
Review the following requirements generated for FridgeChef:

{previous_requirements}

1. Tear these requirements apart. What could go wrong? (e.g., What if the AI suggests a recipe using a spoiled ingredient? What about privacy of photos?)
2. Rewrite the requirements into a Final Technical Spec that addresses these risks.
"""

response_4 = get_completion(prompt_4)
print(response_4)

# FINAL RESULT:
# The output now includes "Liability Disclaimers" for food safety,
# "Privacy Masking" (in case the photo captures a prescription bottle in the fridge),
# and "Offline Caching".
# This is a robust, **enterprise-grade PRD**.
```

You can submit the code (ipynb or py) and the PRD it created (as text or doc or pdf file).

I need you to create a comprehensive Product Requirements Document (PRD) for my product idea. Please read the DOCX skill documentation first to ensure the document follows professional formatting standards.

Product Information

Product Name: [Your product name]

Product Vision/Idea: [Describe your core product idea in 2-4 sentences]

Problem Statement: [What problem does this solve? Who experiences this problem?]

Target Users: [Who are your primary and secondary users? Include demographics, behaviors, pain points]

Key Features/Capabilities: [List 5-10 main features or capabilities you envision]

Success Metrics: [How will you measure success? Include KPIs, goals, metrics]

Constraints & Assumptions: [Any budget limits, technical constraints, timeline requirements, or key assumptions]

Competitive Context: [Known competitors or alternatives, if any]

Additional Context: [Any other relevant information about market, technology, business model, etc.]

PRD Requirements

Please create a complete, professional Product Requirements Document that includes:

1. Executive Summary

- Brief overview of the product

- Key objectives and success criteria

- High-level scope

2. Product Overview

- Vision and strategy

- Problem statement with supporting context

- Value proposition

- Target market and user personas (create detailed personas based on the target users provided)

3. Goals and Objectives

- Business objectives

- User objectives

- Success metrics and KPIs

- Timeline and milestones

4. User Stories and Use Cases

- Primary user flows (at least 3-5 detailed scenarios)

- Edge cases and alternative flows

- User journey maps

5. Functional Requirements

- Detailed feature specifications (organized by priority: Must-Have, Should-Have, Nice-to-Have)

- User interface requirements

- Data requirements

- Integration requirements

6. Non-Functional Requirements

- Performance requirements

- Security requirements

- Scalability requirements

- Accessibility requirements

- Compliance and regulatory requirements

7. Technical Considerations

- High-level architecture approach

- Technology stack recommendations

- Third-party dependencies

- APIs and integrations

8. Design Requirements

- UX principles and guidelines

- Key user interface elements

- Accessibility standards

- Branding considerations

9. Risk Assessment

- Technical risks
- Market risks
- Resource risks
- Mitigation strategies

10. Open Questions and Assumptions

- Unresolved questions
- Key assumptions being made
- Items requiring further research

11. Appendix

- Glossary of terms
- References
- Supporting research or data

Formatting Instructions

- Create this as a professional Word document (.docx)
- Use clear heading hierarchy (Heading 1, Heading 2, Heading 3)
- Include a table of contents
- Use tables where appropriate for organizing information

- Make the document comprehensive but scannable

- Use professional business language

- Ensure all sections flow logically

Please create this PRD now and save it as a Word document.

This is a **comprehensive and innovative** Product Requirement Document (PRD) for a **Proof-of-Human-Activity (PoHA) Cryptocurrency System**. Your approach to replacing traditional mining with human activity verification is both **sustainable and inclusive**, addressing key pain points in the crypto space.

Key Strengths of Your PRD

- **Clear Problem Statement:** You've identified the inefficiencies of traditional mining and proposed a human-centric alternative.
- **Well-Defined Goals & Metrics:** Success criteria are specific and measurable (e.g., 99% valid activity verification, < 2s validation).
- **Modular Architecture:** The system is broken down into logical components (task management, sensor capture, verification, rewards, ledger).
- **Security & Privacy:** You've addressed critical concerns like end-to-end encryption, anti-replay, and GDPR/HIPAA compliance.
- **Scalability:** The design supports 10k+ users and dynamic difficulty adjustment.

Next Steps: How to Proceed

1. Code Generation Prompts (Per Module)

To turn this PRD into a working prototype, you can generate code prompts for each module. Here's how you might structure them:

A. Task Management Module

Prompt:

“Generate a Node.js/Express API for task management in a PoHA cryptocurrency system.

- Endpoints: POST /tasks, GET /tasks/{task_id}
- Task schema: {task_id, task_type, duration_sec, conditions, created_at}
- Ensure tasks are time-bound and cannot be reused.
- Use MongoDB for storage.”

B. Sensor & Activity Capture

Prompt:

“Generate a Python script to simulate sensor data capture for a PoHA system.

- Simulate motion, heart rate, and focus metrics.
- Preprocess raw data into statistical summaries and encoded strings.
- Output: {user_id, task_id, raw_activity_data, activity_hash, timestamp}”

C. Verification Engine

Prompt:

“Generate a Python class for activity verification in a PoHA system.

- Input: submission object (user_id, task_id, activity_hash, timestamp)
- Verify: task validity, hash authenticity, no double rewards.
- Use SHA-256 for hashing.
- Return: {status: ‘valid’|’invalid’, reason}”

D. Blockchain Ledger

Prompt:

“Generate a simple blockchain ledger in Python for a PoHA system.

- Block schema: {index, prev_hash, submission_hash, reward_to, reward_amount, timestamp, block_hash}
- Ensure immutability and prevent double spending.”

---

2. Frontend UI Spec

You can draft a UI spec for a **miner dashboard** and **admin panel**:

- **Miner Dashboard:** Task list, activity tracker, reward history, wallet balance.
- **Admin Panel:** Task creation, verification logs, user management, ledger explorer.

---

3. Whitepaper Adaptation

To turn this into a whitepaper:

- **Abstract:** Summarize the innovation and benefits.
- **Technical Deep Dive:** Explain the verification algorithm, hashing, and ledger.
- **Economic Model:** Tokenomics, reward scaling, and inflation control.
- **Use Cases:** Fitness, healthcare, corporate wellness, etc.

---

4. Industry-Specific Adaptations

- **Oil & Gas:** Tasks could include safety drills, equipment inspections, or physical training.
- **Medical:** Reward patients for adherence to rehab exercises or wellness activities.

---

5. Portfolio Project Packaging

To showcase this as a portfolio project:

- **GitHub Repo:** Organize code by module, include README with setup instructions.
- **Demo Video:** Walk through the miner experience and admin verification.
- **Live Demo:** Deploy a sandbox version (e.g., on Vercel/Heroku).

Which path interests you most?

- Should we start with **code generation prompts** for a specific module?
- Would you like a **UI spec** or **whitepaper outline** next?
- Or, do you want to explore **industry adaptations** or **portfolio packaging**?